

Stanford RNA 3D Folding

Model integration and RNA candidate processing

Yuzhen Guan (Alex) | Team GZSL | September 2026

COMPETITION RESULT	TASK	REPOSITORY
0.42295 private 76 / 1,516 teams Bronze medal [S1,S2]	Five candidate structures per RNA; one C1' coordinate per nucleotide	Competition notebooks, technical notes, and CPU utilities

About the project

I competed in Stanford RNA 3D Folding as part of Team GZSL. My work focused on coordinating experiments, analyzing results by sequence length, and integrating predictions from pretrained models. We finished 76th out of 1,516 teams with a final private leaderboard score of 0.42295.

Our approach combined Protenix, DRfold2 and a pipeline recorded in my notes as trRNA. The practical questions were how to allocate inference work across sequence lengths, how to choose five candidate structures, and how to keep every predicted coordinate matched to the right nucleotide.

Results

Evaluation	Score	Context
Final private leaderboard	0.42295	Team GZSL, rank 76; bronze.
Later public test	0.409	Three-model ensemble result recorded in my solution notes.
Development comparison	0.334 to 0.368	Retained report; the exact validation split is no longer clear.

I keep these results separate because they come from different evaluation contexts. The public/private difference is not a same-split improvement. Detailed experiment records are in docs/RESULTS.md.

Why I maintain this repository

I brought together the inference backups I still have and a small package for working with predicted coordinates. I added the CPU utilities after the competition so that the data checks, candidate handling and segment alignment can be run independently of a model installation.

The original folding models remain upstream dependencies. This repository runs the coordinate-processing workflow and synthetic examples; recovering the exact scored GPU run still requires the matching model assets and final submission CSV.

Model integration

What is in the retained inference notebooks

My solution notes describe length-dependent preferences for trRNA, DRfold2 and Protenix. The two retained notebooks capture separate experiments from that work: Protenix with trRNA candidate replacement, and DRfold2 inference. I keep their actual settings visible instead of treating them as one final submission script.

Protenix + trRNA backup

- Five Protenix candidates
- PyRosetta ordering
- For ≤ 300 nt: replace slot 5 with trRNA candidate 1

DRfold2 backup

- Length-dependent checkpoints
- 480 nt windows for long inputs
- Order by distance to mean score
- Keep five candidates

Protenix with trRNA candidate replacement

The notebook uses Protenix 0.4.6, model_v0.2.0 weights, 10 cycles, 200 diffusion steps and seed 42, with MSA disabled. It sorts five Protenix candidates by PyRosetta score. For targets up to 300 nt, it retains the first four and replaces slot 5 with the first trRNA candidate. The structures stay separate; this step does not average their coordinates.

DRfold2 candidate generation

The DRfold2 backup uses fp32 and cfg_97. Its checkpoint counts are 20, 10, 5 and 5 for lengths ≤ 100 , 101-200, 201-480 and >480 nt. With GET_CENTER=True, candidates are ordered by distance from the mean geometric heuristic score, rather than minimum energy.

Long sequences

The Protenix backup predicts up to 960 nt and pads the remaining coordinates with zeros. DRfold2 uses 480 nt windows with a one-residue overlap and truncates beyond 2400 nt. These shortcuts leave incomplete structural predictions. In the current utilities I require full residue coverage and enough overlap to determine a rigid alignment.

Status of the backups

The Protenix/trRNA notebook retained outputs for 12 Protenix targets and nine trRNA replacements before a final concatenation error. The DRfold2 notebook has no saved outputs. I documented dependencies and known issues in docs/HISTORICAL_NOTES.md. The two Python backups use a different Protenix configuration: 400 diffusion steps and seed 0.

The retained files contain inference code. Training records for the fine-tuning mentioned in my earlier notes are missing, as are the exact historical trRNA source snapshot and the final scored submission.

Coordinate processing

The CPU package and its design choices

I use the current Python package to inspect, assemble and export coordinates produced by external predictors. Separating this work from neural inference makes the data contracts easier to check and keeps the basic workflow runnable on a laptop.

Component	Implementation	Purpose
Input validation	Check target IDs, residue order, finite values and array shapes.	Keep each coordinate tied to the correct nucleotide.
Window assembly	Kabsch alignment on corresponding overlap residues; equal blending after alignment.	Bring independently predicted windows into a common frame.
Candidate selection	Round-robin across model pools; energy ordering within declared comparable groups.	Retain multiple sources without mixing incompatible energy scales.
Submission export	Five distinct candidate IDs and the 18-column CSV schema.	Catch incomplete candidate sets and malformed submissions.

Routing and window boundaries

The current planner uses the model labels `trrosettarna` for 1-100 nt, `drfold2` for 101-480 nt, and `protenix` above 480 nt. These defaults follow the written strategy; the notebooks use their own policies. The `trrosettarna` label is a working name for the tRNA route, whose exact upstream snapshot is unresolved.

Default windows are 300 nt long with 50 nt overlap. The first window anchors the coordinate frame. Each additional window needs at least three non-collinear corresponding points; gaps and ambiguous fits are rejected. Averaging the aligned overlaps does not resolve long-range contacts between distant windows.

Selection and evaluation

Different candidate IDs or model sources do not guarantee different folds. Round-robin selection is a transparent baseline, and energy values are only compared within explicitly compatible groups. Structural diversity and best-of-five accuracy need to be measured on real targets.

For evaluation, the organizer reference compares candidate structures with native conformers and aggregates the best comparisons by target. The optional local `USalign` helper evaluates one PDB pair. It does not implement the complete competition scoring procedure. [S3]

Running the project

Reproducibility, limitations and next experiments

After cloning the repository, I recommend a fresh Python environment and a normal package installation. The default workflow needs CPU and NumPy; it does not download model weights or competition data.

```
python -m pip install .
python scripts/verify_repo.py --output outputs/verification.json
```

The verifier runs 45 tests and 11 command checks, including the installed package outside the checkout, input export, submission import and numeric roundtrip checks. The synthetic demo contains three targets and produces 752 rows, 18 columns and five candidates per target. Source hashes check that the installed package matches the checkout.

The GitHub workflow checks Windows and Ubuntu with Python 3.10 and 3.12. Its Ubuntu / Python 3.12 job also executes the seven-cell walkthrough in a Jupyter kernel. I link each run and its downloadable report so readers can inspect the results for a specific commit.

[Installation and verification guide](#) | [GitHub Actions](#) | [Input formats and commands](#)

What I would evaluate next

With matching model assets and a fixed allowed-data split, I would compare a common-model baseline with length routing, then compare selection policies on the same saved candidate pool. For long sequences I would report both fold quality and boundary geometry, alongside runtime, GPU memory and complete-target coverage. These are proposed experiments, not completed benchmarks.

References and project records

[S1] [Kaggle award certificate](#): Alex GUAN, 76 / 1,516 teams, bronze, September 24, 2025.

[S2] [Final leaderboard screenshot](#): Team GZSL, 0.42295. My [results notes](#) separate this private result from the later public score of 0.409.

[S3] [DasLab competition tools](#) and [TM-score reference](#); USalign.

[S4] Upstream models: [Protenix](#), [DRfold2](#). See [references](#) for tRNA-related components and attribution.

[S5] [Retained inference notebooks](#); [dependency and implementation notes](#); [source file inventory](#).

This September 2026 write-up brings together my competition notes and the current repository. The original upstream model authors retain credit for their architectures and implementations.